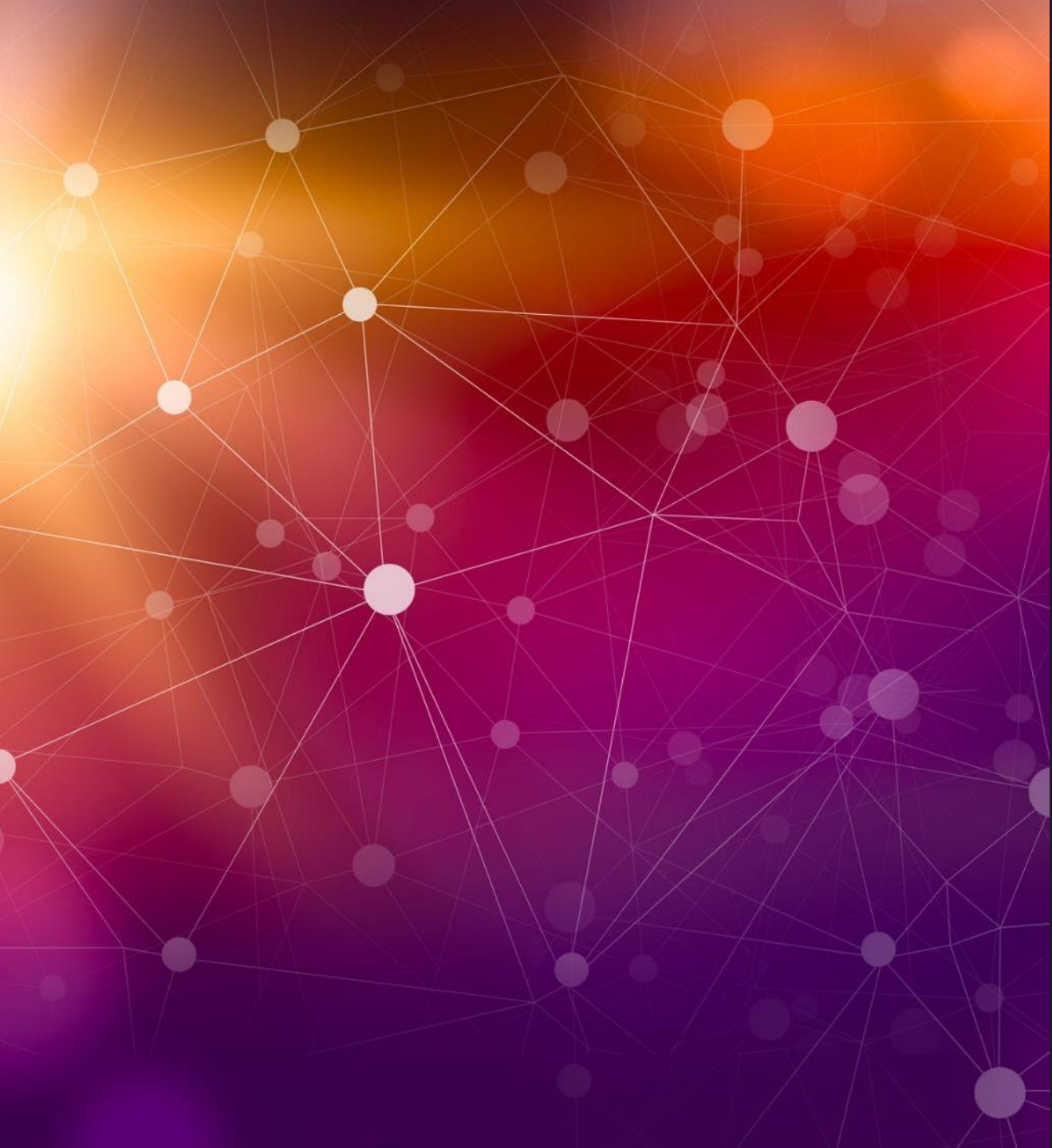




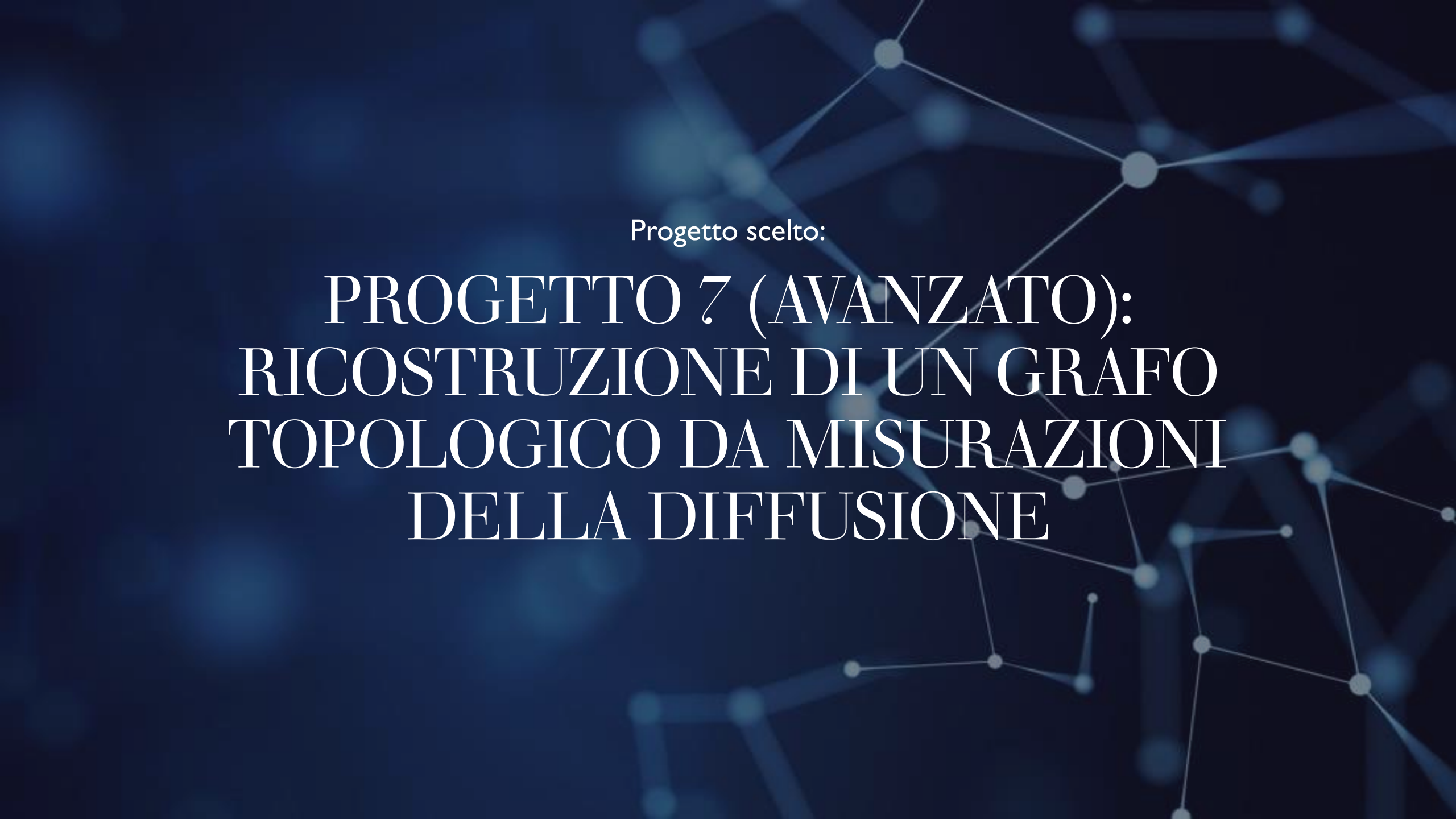
Presentazione progetto Calcolo Numerico

Gruppo: Paduret Alessandro, Singh Alex, Stupu David, Valente Ivan

A.A. 2025-2026

Corso di Laurea Triennale Informatica
Dipartimento di Matematica, Scuola di Scienze

4 febbraio 2026



Progetto scelto:

PROGETTO 7 (AVANZATO): RICOSTRUZIONE DI UN GRAFO TOPOLOGICO DA MISURAZIONI DELLA DIFFUSIONE

Sommario

- Contesto teorico e scopo del progetto
- Inizializzazione
- Problema 1: dati diffusivi sovrabbondanti
- Problema 2: dati diffusivi lacunosi
- Considerazioni finali





Contesto teorico e scopo del progetto

Scopo del progetto e Laplaciano

- Grafo topologico: insieme finito di «nodi» e di una relazione anti riflessiva simmetrica non transitiva su di essi
- Dato un grafo di cui non si conosce la topologia, la si può dedurre attraverso le proprietà che ne descrivono il cambiamento nel tempo $\rightarrow$ obiettivo: risoluzione di un problema inverso
- Risoluzione tramite applicazione dell'operatore Laplaciano «L» che generalizza la derivata seconda sui grafi
- Dato $G = (V, E)$ con m nodi, $L = D - A$, dove «A» è la matrice delle adiacenze e «D» la matrice dei gradi
- Per una funzione $u: V \rightarrow R$ sui nodi, l'operatore Laplaciano $(Lu)_i = deg(i)u_i - \sum_{i \sim j} u_j = \sum_{j \sim i} (u_i - u_j)$ quantifica lo squilibrio tra u_i ed i suoi vicini
- Se $(Lu)_i > 0$ allora u_i è maggiore della sua media locale, e la diffusione farà diminuire u_i , l'opposto per < 0



Evoluzione dell'informazione

L'evoluzione temporale di $u(t)$ è descritta dall'equazione differenziale

lineare: $\frac{d}{dt}u(t) = -Lu(t), t > 0$

tale che per condizione iniziale $u(0) = u_0$, abbiamo come soluzione

analitica: $u(t) = e^{(-tL)}u_0(?)$



Lo riempiremo nella prossima slide



Dallo spazio dei nodi allo spazio dei modi

- Dato che L è reale e simmetrica (per grafi non orientati), grazie al Teorema Spettrale, essa è diagonalizzabile nella forma: $L = P\Lambda P^T$
- Λ è una matrice diagonale tale che $0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_m$ e «P» contiene gli autovettori ortonormali ϕ_k come spazio delle colonne
- Sostituendo nella soluzione analitica di prima (?), otteniamo che

$$u(t) = \sum_{k=1}^m (u_0^T \phi_k) e^{-\lambda_k t} \phi_k u(t) = P \cdot e^{-\Lambda t} \cdot P^T \cdot u(0)$$

- ϕ_k rappresenta i modi stazionari
- λ_k rappresenta la velocità di decadimento, con $\lambda_1 = 0$ significando stato di equilibrio, mentre λ_k grandi significano decadimento veloce



Esperimenti e casi

- Non conoscendo L , P , o Λ , ed avendo $(u(j)(0), u(j)(t_k))_{j=1\dots M, k=1\dots T}$, ovvero coppie di input ed output, la risoluzione effettiva si divide per due sottocasi fondamentali:
 - 1) Dati sovrabbondanti (caso $M \geq m$): esperimenti con condizioni iniziali linearmente indipendenti. In questo caso possiamo direttamente stimare la dinamica con il rapporto incrementale discretizzato e approssimando $L \approx -\dot{U}U^+$
 - 2) Dati lacunosi (caso $M < m$), il caso in cui il problema è mal posto (sistema sottodeterminato). Possiamo sfruttare lo spazio di Krylov grazie al quale avendo anche solo una condizione iniziale u_0 , permette di dedurre l'evoluzione esplorando il grafo:
 $u(t1) = e^{(-t1L)}u0, u(t2) = e^{(-t2L)}u0, \dots$
- Se riusciamo a raccogliere abbastanza istanti temporali t_k , possiamo rendere i vettori $u(t_k)$ linearmente indipendenti costruendo una matrice delle osservazioni “estesa”
- Raggiungiamo il grado minimo del polinomio per L anche con pochi esperimenti, a patto che si campioni abbastanza a lungo nel tempo



Rapporto incrementale e soluzioni

- La realizzazione del rapporto incrementale discreto è fondamentale per risolvere entrambi i problemi, dato che è regressione banale basata su due dati (t_0 e Δt), l'approssimazione dell'operatore avviene come $L \approx -\dot{U}U^+$,
- Ovvero, in pratica, come $L \approx -(U\{2\} * c - I)/dt$,
- $dt = t_1 - t_0$ e $c = U(0)^{-1}$ se $M = m$, altrimenti sfruttiamo la pseudo-inversa di Moore-Penrose U^+ (dato che $M \geq m$).





Inizializzazione

buildsyntheticdata.m

- Parametri del problema: numero di nodi, numero di esperimenti, numero di istanti temporali, tempo finale di osservazione, ordine di grandezza del rumore, probabilità di creare un arco
- Generazione del grafo e del Laplaciano
- Generazione dei dati iniziali ($u(0)$)
- Simulazione della diffusione dei vari esperimenti fino al tempo finale
- Snapshots del grafo (le varie $u(\dots)$) per ogni istante temporale stabilito
- Salvataggio dei dati
- Vediamo il codice...

Tramite uso delle «cells» in MATLAB





Problema 1: dati diffusivi sovraabbondanti

Spiegazione del problema

Data la lista degli m nodi $V = \{v_1, \dots, v_m\}$, una serie finita di istanti temporali $0 = t_0 < t_1 < \dots < t_n = T$ ed la misurazione di $M \geq m$ processi diffusivi $U\{j\}(i, k) := u^{(k)}(v_i, t_j) + \epsilon_{i,j,k}$, con $i = 1, \dots, m, j = 1, \dots, n$ e $k = 1, 2, \dots, M$, e dove $\epsilon_{i,j,k}$ è un errore di misurazione, trovare la matrice di adiacenza A del grafo.

- Sistema lineare sovradeterminato: $Ax = b$ con $A \in M_{M \times m}$ con $M \geq m$
- Non esiste una soluzione esatta x che soddisfi $Ax = b$
- Obiettivo: cercare la x migliore possibile
- Strumento utilizzato: minimi quadrati con fattorizzazione SVD per minimizzare la norma del residuo $\|Ax - b\|_2$



solutore_1.m

- Risolve il problema I
- «A» = cells «U», «x» = «c», «b» = quale nodo prendiamo come partenza
- Calcolo delle traiettorie tramite calcolo del vettore «c» (con «pinv» per SVD) e delle derivate (velocità di diffusione)
- Costruzione della matrice di adiacenza (e di conseguenza del grafo) grazie a clustering (con «kmeans» in MATLAB)
- Vediamo il codice...
- Analizziamo quanto è stato efficiente il processo tramite «statistiche.m»...
- Visualizziamo graficamente i fenomeni grazie a «visualizza_dati.m»...





Problema 2: dati diffusivi lacunosi

Spiegazione del problema

Data la lista degli m nodi $V = \{v_1, \dots, v_m\}$, una serie finita di istanti temporali $0 = t_0 < t_1 < \dots < t_n = T$ ed la misurazione di $M < m$ processi diffusivi $U\{j\}(i, k) := u^{(k)}(v_i, t_j) + \epsilon_{i,j,k}$, con $i = 1, \dots, m, j = 1, \dots, n$ e $k = 1, 2, \dots, M$, e dove $\epsilon_{i,j,k}$ è un errore di misurazione, trovare la matrice di adiacenza A del grafo.

- Sistema lineare sottodeterminato: $Ax = b$ con $A \in M_{M \times m}$ con $M < m$
- Infinite soluzioni x che soddisfano esattamente $Ax = b$
- Obiettivo: cercare la x migliore possibile
- Strumento utilizzato: soluzione di minima norma con fattorizzazione SVD per cercare la x che ha la norma $\|x\|_2$ più piccola tra tutte le soluzioni valide



solutore_2.m

- Risolve il problema 2
- «A» = cells «U», «x» = «c», «b» = quale nodo prendiamo come partenza
- Calcolo delle traiettorie tramite calcolo del vettore «c» (con «pinv» per SVD) e delle derivate (velocità di diffusione)
- Problema: ci sono meno esperimenti rispetto ai nodi → soluzione: aumentiamo il numero di esperimenti artificialmente, utilizzando ogni istante temporale come un nuovo esperimento
- Costruzione della matrice di adiacenza (e di conseguenza del grafo) grazie a clustering (con «kmeans» in MATLAB)
- Vediamo il codice...
- Analizziamo quanto è stato efficiente il processo tramite «statistiche.m»...
- Visualizziamo graficamente i fenomeni grazie a «visualizza_dati.m»...



solutore_sliding.m

- Risolve sempre il problema 2, con la stessa logica di base, ma in maniera più robusta
- Migliore gestione del rumore:
 - Il precedente codice calcolava la derivata guardando solo due istanti consecutivi → derivata molto sensibile al rumore
 - Il nuovo codice prende in esame una finestra temporale, in cui esegue una regressione lineare per trovare la pendenza media → derivata molto più stabile
- Costruzione della cell « U »:
 - Il precedente codice concatenava gli snapshot dei singoli istanti ($U\{1\}, U\{2\}, U\{3\}, \dots$)
 - Il nuovo codice concatena i risultati delle finestre, estraendo media delle temperature e velocità di variazione calcolata con la regressione
- Le derivate vengono calcolate per ogni finestra subito, prima ancora di sapere i coefficienti di « c ».
- « c » viene calcolato basandosi sugli stati medi.
- Vediamo il codice...
- Analizziamo quanto è stato efficiente il processo tramite «statistiche.m»...
- Visualizziamo graficamente i fenomeni grazie a «visualizza_dati.m»...





Considerazioni finali

Considerazioni finali

- Come esplicitato su Moodle, era possibile contattare il prof. Piazzon per suggerimenti e dubbi
→ «solutore_sliding.m» perché ci interessava migliorare «solutore_2.m»
- Scelta di questo specifico progetto dovuta a:
 - Conoscenze pregresse sui grafi dal Corso del Primo Anno «Algebra e Matematica Discreta»
 - Progetto più interessante di tutti (a nostro parere)
 - Ambizioni future





FINE

Grazie dell'attenzione!